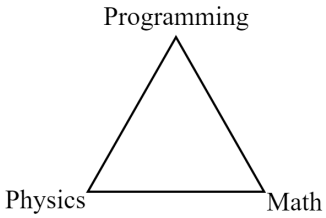
Lecture 3: Memory and Buses

Lui Holder-Pearson

July 17, 2026

Learning Goals

- Understand rudimentary logic functions and logic gates.
- Understand how sequential logic components are used to implement computer memory.
- Understand and interact with the arrangement of memory within embedded systems.
- Know what pointers within C are, and how they pertain to memory arrangement.
- Briefly understand how information is transmitted within an embedded system.



Further Reading

ENCE361 - Next year
https://www.youtube.com/watch?v=fpnE6UAfbtU (Nice visual tools for logic gates)
https://www.youtube.com/watch?v=QzWW-CBugZo (Amazing visualisation of buses)

References

Ciaran Moore's past ENCE260 lecture notes
Rizzoni and Kearns. *Principals and Applications of Electrical Engineering*, Chapters 13 & 14.

Version: 1.1

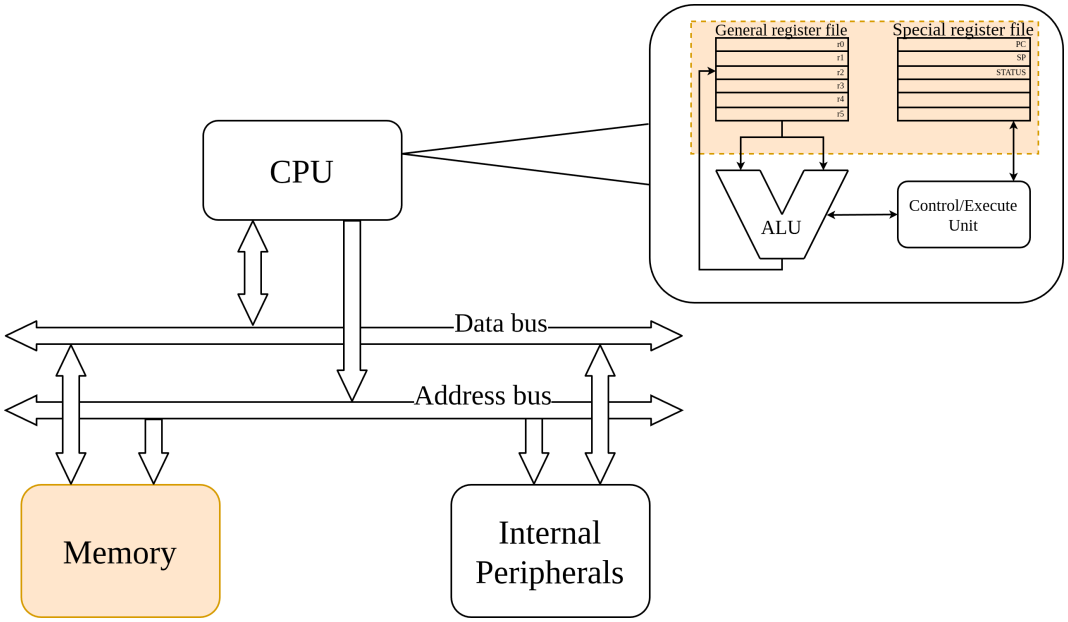


Figure 1: The aspects of a computer which are comprised of some form of memory.

Memory is found throughout embedded systems. It allows:

- Recording of data outputs.
- Storing intermediary results.
- Storing the programme itself.

How is memory implemented? How does the implementation impact our interaction?

Recall:

Computers are digital devices. This means that at a fundamental level, the smallest amounts of information they can deal with can only ever be one of two states: binary systems. We will be discussing boolean variables, which take a value of either `TRUE` or `FALSE`.

To encode these mathematically, `FALSE` = 0, and `TRUE` = 1

Representation		
Numerical value:	0	1
Logical value:	`FALSE`	`TRUE`
Voltage:	Low	High
Presence:	Absent	Present
Switch state:	Open	Closed.

1 Logic functions

Memory is made of some sort of logic circuit. Logic, or boolean logic, is the logic that computers are able to perform in hardware. Logic functions are similar to numerical functions that we are reasonably familiar with, but only accept binary inputs and outputs. Thus, instead of being defined mathematically, they are defined logically.

Boolean logic: the mathematics of binary, will be discussed further in Lecture 4.

For logic functions, truth tables are used to explain the output as a function of the different inputs:

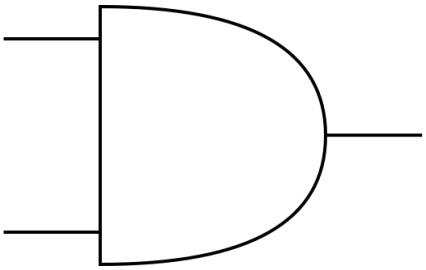
I_1	I_2	O

I_1	I_2	O

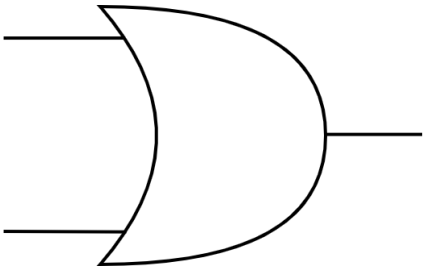
I	O

I_1	I_2	O

Logic circuits are made up of logic gates:



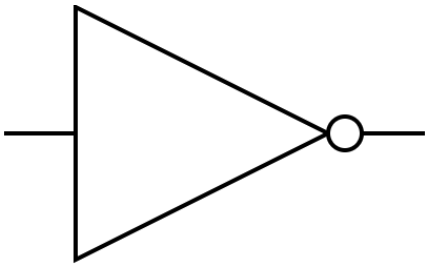
(a) AND gate.



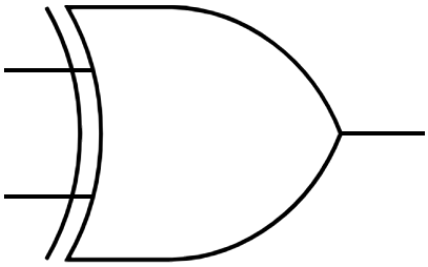
(b) OR gate.

AND and OR gates are the most common, and combinations of these basic gates can achieve powerful results. These gates are known as **combinational logic**, as the output is dependent on the inputs only: A given combination of inputs will always produce the same output.

It is possible to use combinations of logic gates to perform more complex tasks.



(a) NOT gate.



(b) XOR gate.

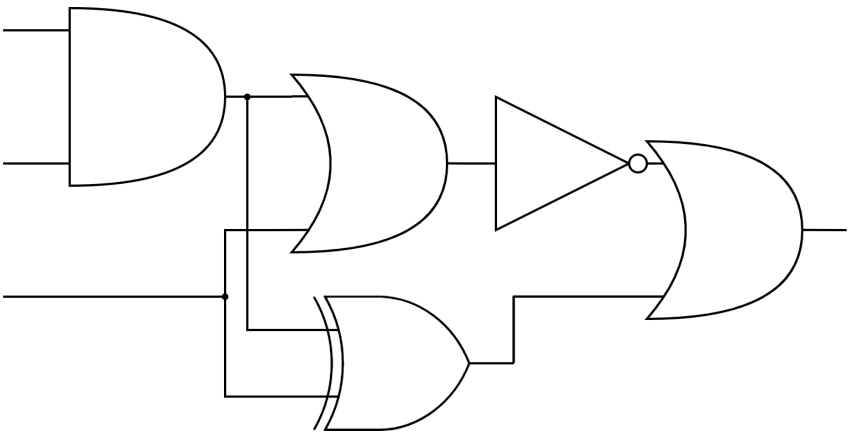
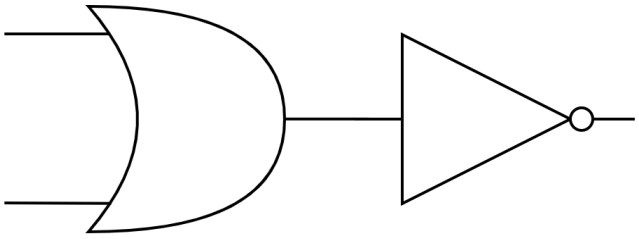


Figure 4: Example of a more complex logic circuit.

1.1 Sequential logic

Where basic logic gates are combinational only, we can use series of logic gates to give the system some sort of memory. The basis for this is the NOR gate:

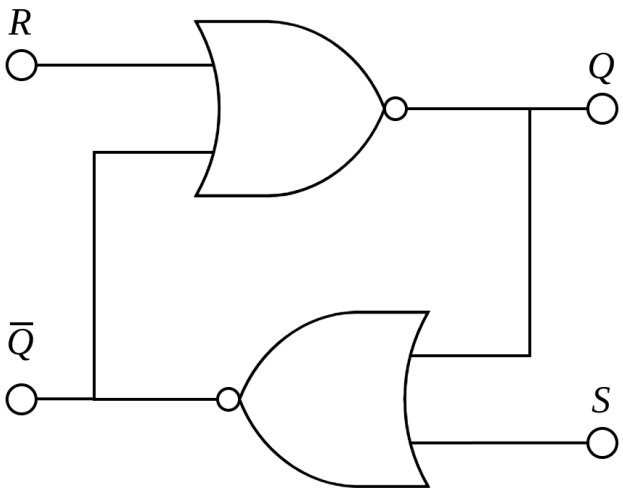


I_1	I_2	OR	NOR

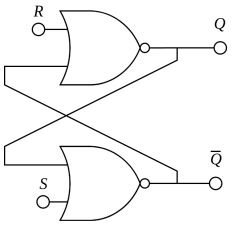
It is common to combine the NOT logic gate with other gates, so the NOR gate is drawn:

SR Latch

The SR latch is the simplest form of sequential circuit.



Alternatively drawn:



We write the outputs of the circuit Q and $\overline{Q}$, because they must always be different.

Inputs		Outputs			
S	R	Q_k	$\overline{Q_k}$	Q_{k+1}	$\overline{Q_{k+1}}$
0	0	1	0		
0	0	0	1		
0	1	1	0		
0	1	0	1		
1	0	1	0		
1	0	0	1		

$\rightarrow$ *The output of the circuit depends on the previous state: Sequential logic.*

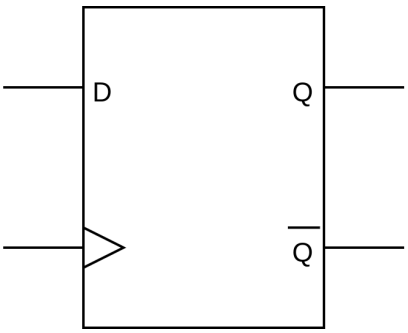
We can use an SR latch to store an individual bit of memory.

However, SR latches are imperfect: when does the output of the SR latch change?

D flip flop

Through the introduction of a control line, we are able to provide an instruction to 'listen' to the input. This forms what is known as a flip flop, which is the fundamental smallest unit of storage within computers.

Input	Current		Next	
D	Q_k	$\overline{Q_k}$	Q_{k+1}	$\overline{Q_{k+1}}$
0	0	1	0	1
0	1	0	0	1
1	0	1	1	0
1	1	0	1	0



The introduction of a clock input allows the synchronisation of multiple devices. The clock, or `CLK` effectively provides the instruction to the D flip flop to capture the input of the D signal, and store it:



2 Memory

By using many, many, many flip flops, we can make some physical memory which is able to electronically controlled with digital signals. Each array of flip flops is called a register.

2.1 Registers

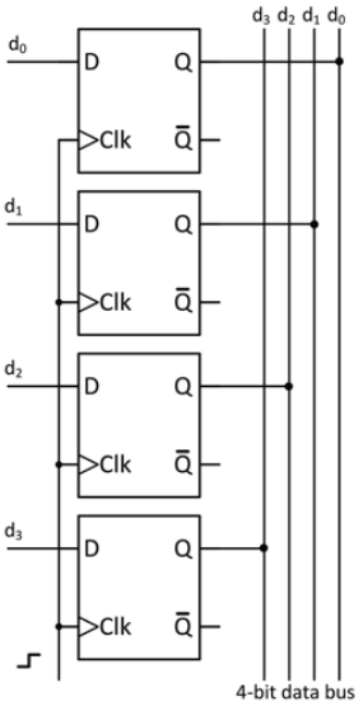


Figure 6: Example of a 4-bit register made of D flip flops.

The ATMega32u2 has 8-bit registers in general, but some pairs of 8-bit registers have the special capability to act as 16-bit registers.

- **Bit**: The smallest amount of information able to be stored; one individual digit which takes the value of either 0 or 1.
- **Byte**: 8 bits. Specifically, eight consecutive bits which are addressed as a single unit.
- **Word**: Less specific, this is the default number of bits that is manipulated by the CPU of a microcontroller. The registers within the CPU are always one word long.
- **Address**: Each byte of memory has a specific address within memory, this is the number (usually in hexadecimal) which says where in memory it is.

As well as flip flops/latches, other physical methods of storing bits is in the charge of a capacitor, magnetic patterns on a disk, or electrons trapped in a transistor gate.

Address	Bit 0	Bit 1	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7
0000								
0001								
0002								
0003								
0004								
0005								

Figure 7: Example of a memory with 8-bit words.

2.2 Addressing

Each byte in memory has its own address. For example, in Figure 8, there are two 8-bit (1 byte) variables, `varA` and `varB`, stored at addresses `100` and `101`.

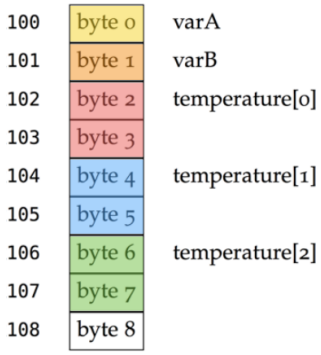


Figure 8: Example of variables stored within memory.

Pointers

In the C programming language, it is possible to manipulate data specifically using the address, rather than the variable name. This is done with the use of *pointers*. Simply put, a pointer is a special variable which stores the *memory address* where the value of the variable is held.

Among other things, pointers allow us to choose a variable depending on decisions made while the programme is running.

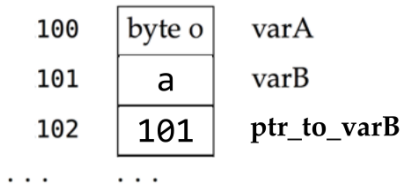
- There are two special characters that C uses to manipulate points: '*' and '&'.
- `*address` — "The value pointed to by `address`".
- `&variable` — "The address of `variable`".
- Like variables, pointers have types (e.g. `uint8_t*`, `char*`) that indicate the type of variable they point to.

Example

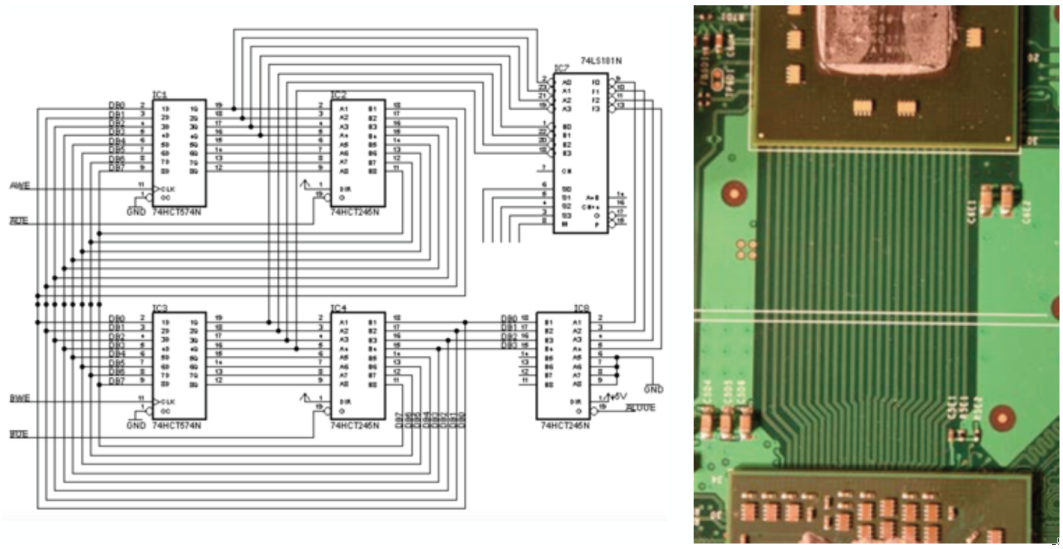
What output does the following code generate?

```
char varB = ‘a’;
char* ptr_to_varB;

ptr_to_varB = &varB;
printf(‘‘%c", *ptr_to_varB);
```



3 Buses



It is important that information is able to be transferred from one place to another. There are several different types of memory that must be accessed:

- **General purpose registers**: These are the parts of memory that are actively manipulated by the CPU. They are extremely physically close to the CPU to enable fast access.
- **RAM**: Easily-modifiable memory, fast to read and write.
- **Program memory**: Memory where the program is stored. Often flash memory. Fast to read, slow to write.
- **EEPROM**: Non-volatile memory.

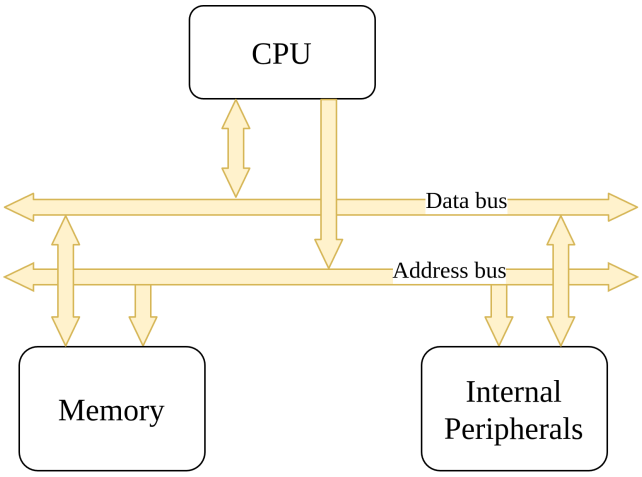


Figure 9: Computer architecture model highlighting the buses.

These different kinds of memory have different physical locations, for example Figure 10 shows the layout of the ATMega32u2.

Question: can you highlight each of the above memory types in Figure 10

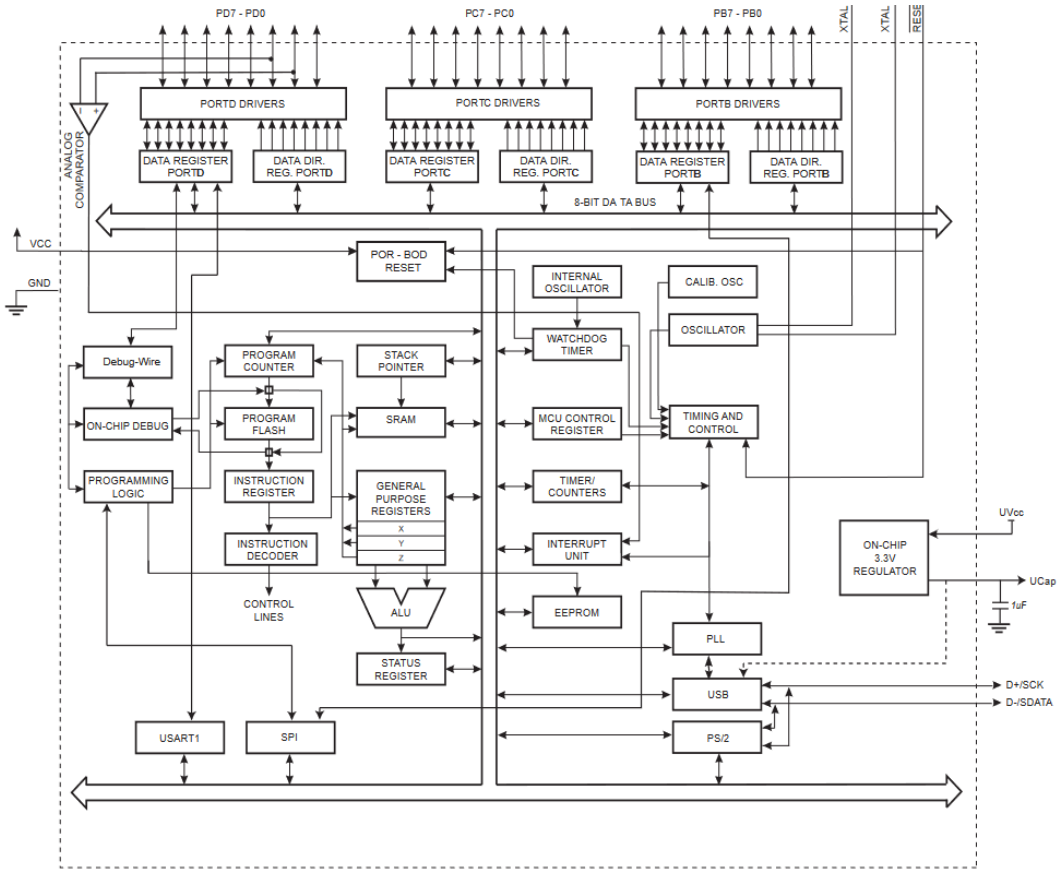
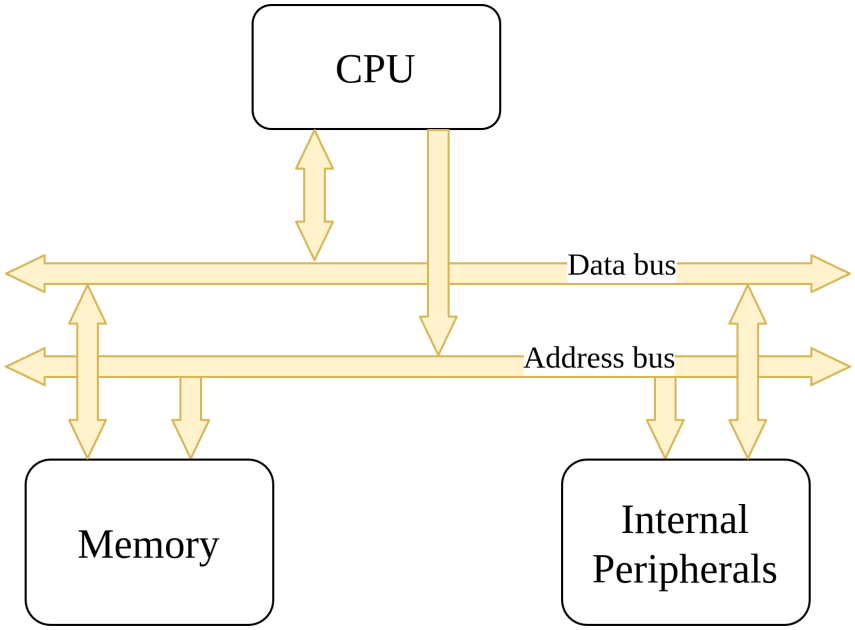


Figure 10: Functional block diagram of the ATMega32u2

The bus width is simply the number of individual connections within the bus. In microcontrollers like the ATMega32u2, this is almost always equal to the word size, and thus the general purpose register size. Therefore, it is generally possible for most sub-systems connected to the bus to be written from the general purpose registers, and to be read into the general purpose registers.



3.1 Types of bus

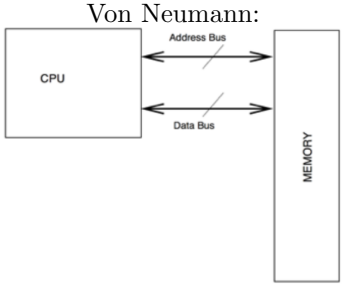
Within a device such as the ATMega32u2, there are several different buses and bus types.

1. Data bus:
   - Carries the actual data: transports information between the CPU and memory.
   - Width equal to CPU word length.
   - Bidirectional.
2. Address bus:
   - Specifies memory locations.
   - Address space is $2^{buswidth}$.
   - Controlled by the CPU.
3. Control bus:
   - Specifies the direction of data communication over the databus. This is done from the point of the CPU, so `READ` mode is being read into the CPU, and `WRITE` mode is writing from the CPU.
   - Specifies when data transfer occurs over the data bus.

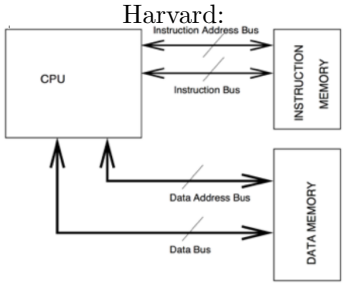
There is variation in architectures where the programme memory is stored in the same physical memory as data memory (Von Neumann architecture), or in a separate memory (Harvard architecture). Harvard architecture has become increasingly dominant since the early 2000's to the extent that Von Neumann now only receives a small mention at the bottom of page 12 of lecture notes, despite the increased complexity of Harvard architecture. This is because of the significant gains in performance from being able to retrieve instructions from programme memory while carrying out operations on data.

The limited size address bus is why some older computers that still run a 32-bit operating system are limited in both memory, and storage: $2^{32} = 4,294,967,295$, therefore memory, like RAM, that is addressed by each byte is limited to this number of bytes. For storage, where addressable 512-byte sectors are used, this is approximately 2.2TB.

Von Neumann:



Harvard:



Summary

- Understand rudimentary logic functions and logic gates:
  → The basic functions `AND`, `ORNOT`, and `XOR` form the basic logic gates, allowing more complex logic circuits.
- Understand how sequential logic components are used to implement computer memory.
  → Sequential logic enables memory through the use of circuits like the SR latch, and the D flip flop, where the output while stay the same except in specific circumstances, thus enacting memory.
- Understand the arrangement of memory within embedded systems.
  → Memory is formed of arrays of flip flops to form registers.
  → There is also memory for storing data variables in RAM.
  → Every individual byte is has its own address, which is a number specifying its location.
- Know what pointers within C are, and how they pertain to memory arrangement.
  → Pointers are a special variable type which store the address of the variable.
  → Pointers are used in C through the use of *, meaning 'the variable at this address' (e.g. `*ptr_to_charB`), and &, which means 'the address of this variable'.
- Briefly understand how information is transmitted throughout an embedded computer.
  → The data bus is a bidirectional bus with width equal to the word size for transferring information.
  → The address bus is used to specify the address which is being read/written.
  → The control bus governs the data bus, in terms of directionality and synchronisation.

Next Lecture: Lecture 4 - Logic design

4 Questions?